GoWarehouse Integration Brief

Ralulu ERP · prepared for discussion with the client · based on the Partners API 2026-09 documentation (sharedhd.gowarehouse.ai/partners/docs/en/2026-09) and the current ERP code. No live API call has been made yet (no token).

1. Summary

GoWarehouse (GW) offers a REST API for products, orders, shipments, inventory, inbounds and returns. Our ERP already has a Shipping Workbench with a **3PL stage**, but that stage is manual today: staff download a packing Excel and later upload a confirmation file, and neither file format is final. The GW API can fill exactly these two gaps.

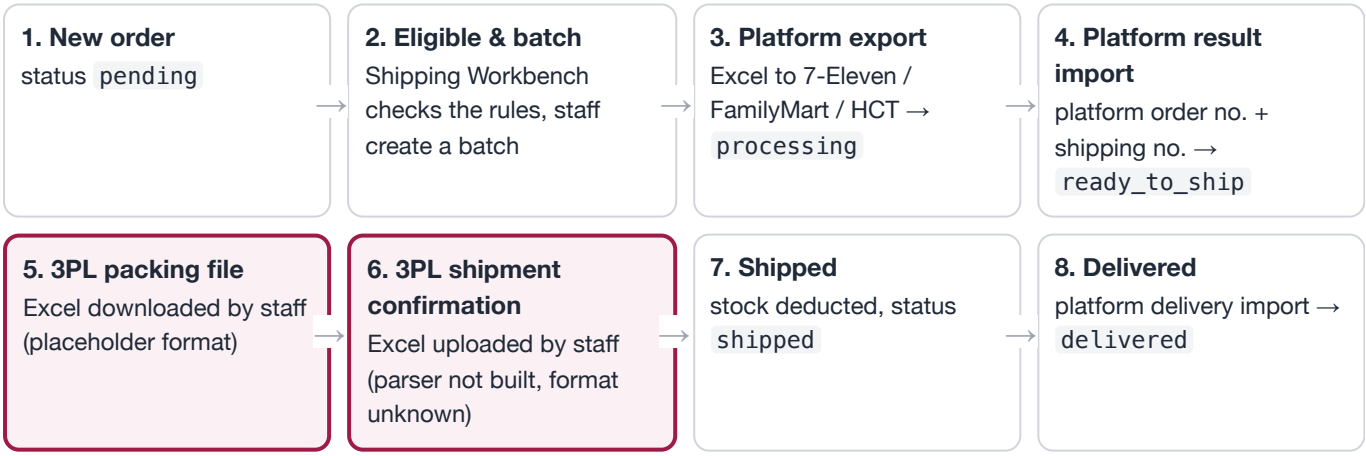
Proposal

1. **Phase 1:** SKU sync (ERP to GW) and a “Send to GoWarehouse” button in the Shipping Workbench batch page, replacing the packing-Excel download for the 3PL.
2. **Phase 2:** pull shipment status and tracking numbers from GW and use them to confirm shipment automatically (replacing the 3PL confirmation upload).
3. **Later / manual for now:** inventory comparison, inbound (from Procurement Orders), returns.

Existing ERP logic (eligibility, batches, platform export and import, stock rules) stays unchanged. The new code only adds an alternative path for the two 3PL steps.

Decisions we need first: (a) whether GW prints the 7-Eleven / FamilyMart / HCT labels we already create, or creates its own carrier shipment; (b) the codes and formats GW expects (section 5).

2. How the ERP works today (from the code)



Highlighted boxes (5 and 6) are where GoWarehouse would plug in.

Eligibility rules (computeEligible in `shipping_batches.go`)

- Order status `pending`, not already in a batch, delivery provider matches (7-Eleven, FamilyMart or courier).
- Created before 00:00 Taiwan time today (UTC+8), so same-day orders wait.
- Kept orders are held until 1 day (shipping date mode) or 4 days (expected delivery mode) before their kept date.
- Each shipment is classified ready / problem / insufficient stock; physical stock is allocated oldest first and never promised twice. Unknown store codes are flagged.

- Orders merged into one shipment are batched as a whole.

Stock model (`stock_calc.go`, `order_status.go`)

- **Actual** (physical stock) only drops when an order moves to `shipped`, which happens when the 3PL shipment is confirmed. Creating, batching, exporting or cancelling never touches it.
- **Ordered** is derived from orders in pending / processing / ready_to_ship, so it cannot drift. Available = Actual + Incoming − Ordered.
- Every status change goes through one function (`applyStatusChange`), so stock, promo, wallet and points side effects stay consistent.

Data we already have

- SKU: `product_variants.sku` (unique, up to 60 characters), one per colour/size variant.
- Order fields: order number, customer name / phone / address, pickup chain and store code, `platform_order_no`, `tracking_number`, shipping fee, `exported_at`, and the keep date with its type.
- Logs: `order_export_log` (what was exported, when) and `shipping_import_log` / issues (what was imported and what could not be matched).

What is manual or unfinished today

- The 3PL packing Excel is a placeholder “until the 3PL confirms its real format” (`excel.js`).
- The 3PL shipment-confirmation and delivery parsers refuse to guess formats (`parsers.js`); the backend rules for applying them already exist.
- Products are created only inside the ERP, with no warehouse catalogue to keep in sync.

3. What we learned from the GW API

Topic	What the documentation says
Access	Base path `/api/partners/2026-09`. Bearer token per integration, with scopes per endpoint (e.g. `products:write`, `orders:write`). The token can be restricted to an IP allowlist.
Limits	180 requests / minute, 10 / second, and 10,000 / day per merchant. Throttled calls return 429 with `Retry-After`. This is comfortably enough for our order volume.
Format	Flat JSON bodies (no wrapper), money and weights as strings, dates `YYYY-MM-DD`, timestamps in UTC, cursor pagination (`page_size`, `since_id`, `has_more`) and `updated_at_min` filters for incremental sync. Errors are machine codes (e.g. `validation_failed`).
Webhooks	9 events: `order.created`, `order.shipped`, `order.cancelled`, `shipment.shipped`, `shipment.status_changed`, `inventory.updated`, `inbound.completed`, `return_receipt.verified`, `return_order.received`. Signed with a secret; requires a public HTTPS endpoint. Polling is a valid alternative.

Endpoints

Area	Endpoints	Notes
Products	`GET/POST /products`, `GET/PATCH/DELETE /products/{id}`	Only `name` and `sku` are required on create. Optional: barcode, brand, category, dimensions, weight, note. SKU is unique per merchant.

Orders	`GET/POST /orders`, `GET/PATCH /orders/{id}`, `POST /orders/{id}/cancel`	Required: `name` (order number) and `items`. Each item needs `sku`, `quantity` and `inventory_type`. Also receiver name, phone, address, `convenience_store_code`, `cod_amount`, `shipping_type`, `warehouse`, `scheduled_shipping_date`, `sales_channel`, `note`. An item `identifier` is echoed back so we can match lines.
Shipments	`GET /shipments`, `GET /shipments/{id}`	**Read-only**: created by the warehouse. Fields: `status` (waiting, shipped, in_transit, arrived_at_hub, arrived_at_destination, received, returned, returned_to_sender, problem), `tracking_number`, `orders` (names, several when merged), `shipped_at`.
Inventory	`GET /inventory_levels`, `GET /inventories`	Per product, warehouse and inventory type: available, pickable, on-hand, allocated and held quantities.
Inbounds	`GET/POST/PATCH /inbounds`, `POST /inbounds/{id}/cancel`	Create needs `name`, `eta` and items. Editable only while `pending`.
Returns	`GET /return_orders`	Read-only (pending, partial_received, received, done, canceled).
Other	Invoices and credit notes, contact addresses, sales channels, attribute definitions, webhook management	Invoices only matter if e-invoices are issued through GW.

Things that affect our design

- If an order contains an SKU GW does not know, GW **silently creates an "UnknownProduct"** and still accepts the order, unless strict mode is enabled for the merchant. So SKU sync must come first.
- `inventory_type` is required on every order item and must exist in GW (the docs mention values like normal, damaged and expired).
- An order can only be edited in GW while it is still `pending`; after that it can only be cancelled (irreversible).
- Shipments cannot be created through the API, so tracking and status always come from GW.
- The API does not expose a field specifically for a carrier tracking number on the order we send. We can pass our platform order number and shipping number in `extra_name` or `note`, but this must be agreed with GW.

4. Recommendation

Priority	Item	Approach
Must	Product SKU sync	ERP to GW only. A "Sync SKUs" button creates missing SKUs (`POST /products`) and updates changed names; a sync log shows what succeeded or failed. Run it before the first order is sent.
Must	Send orders	New "Send to GoWarehouse" button on the batch page, for orders in `ready_to_ship` that were not sent yet. It replaces the packing-Excel download. Result per order is shown (sent / failed + reason).

Strongly recommended	Shipment status and tracking	Poll `GET /shipments` every 10–15 minutes with `updated_at_min` (roughly 100 to 150 calls a day). When GW reports an order shipped, move it `ready_to_ship` to `shipped` through the existing status function, which deducts stock and logs it. Webhooks can replace polling later.
Later	Inventory	Read-only comparison of GW on-hand vs ERP Actual. Do not overwrite ERP stock automatically.
Later	Inbound	Create a GW inbound from a Procurement Order (`po_number`, supplier, ETA, items). Can stay manual at first.
Later	Returns	Read GW return orders; staff still process the return in the ERP. Low volume.
Skip	Invoices, addresses, attributes	Not needed unless GW issues invoices for us.

Proposed order mapping (ERP to `POST /orders`)

GW field	Value from the ERP
`name`	ERP order number (`order_no`)
`items[].sku / quantity`	Variant SKU and quantity of each order line
`items[].identifier`	Our order item ID (comes back in GW events, so lines match one to one)
`items[].inventory_type`	A fixed code from GW (setting, e.g. normal) – to be confirmed
`receiver_name / phone / address`	Customer name, phone and shipping address
`convenience_store_code`	Our validated store code (`pickup_store_code`) for 7-Eleven and FamilyMart
`shipping_type / warehouse`	Mapped from our delivery provider; GW codes to be confirmed
`cod_amount`	Total to pay for cash-on-delivery orders
`extra_name / note`	Platform order number and platform shipping number, if GW needs them to print the existing labels
`scheduled_shipping_date`	Optional: ship date from the keep-order setting

Minimal ERP changes

- One GW client module (token and base URL in settings, never in the repository).
- New columns: `gw_order_id` and `gw_sent_at` on orders, `gw_synced_at` on variants, and one sync-log table.
- Safeguards: an order with a GW id cannot be sent twice; rate-limit and error handling with retry; every send is logged with the user who triggered it.
- Nothing is removed. The Excel path stays available as a manual fallback.

5. Questions for the client and GoWarehouse

Each question has our recommended answer, so the client can simply confirm or correct it.

#	Question	Our recommended answer
1	**Labels:** does GW print the 7-Eleven / FamilyMart / HCT labels created through our platform step, or does it create its own carrier shipment and tracking?	Keep our platform step (least change) and pass the platform order and shipping numbers to GW. If GW creates its own shipments, the platform export step may become redundant and we should redesign. Needs GW's answer first.
2	**Codes:** which `inventory_type`, warehouse name and `shipping_type` codes should we send for 7-Eleven, FamilyMart and home courier?	Ask GW for the exact list. We store them as settings, not in code. We expect `normal` for sellable stock but have not confirmed it.
3	**Store code:** which format does `convenience_store_code` expect?	Our 6-digit store codes; GW to confirm. We can convert if they differ.
4	**Strict SKU mode and sandbox:** can strict mode be enabled, and is there a test merchant?	Yes to strict mode, so unknown SKUs are rejected instead of silently accepted. Test in a sandbox first; if none exists, test with one low-value real order.
5	**Stock ownership:** when does stock go down?	Keep today's rule: ERP Actual drops when the order becomes `shipped` (GW confirmed), and GW stock is shown for comparison only. This avoids deducting twice.
6	**Merged shipments:** when several ERP orders ship together, how should GW receive them?	Send each ERP order separately with the same note reference; GW shipments already list several order names. GW to confirm.
7	**Changes and cancellation:** what happens if a customer cancels after we sent the order?	The ERP already locks orders after export. If GW is still `pending` we call cancel; after that it is a manual case with GW.
8	**Kept orders:** should GW hold orders until the kept ship date?	No. The ERP already holds them and only sends when they are due; `scheduled_shipping_date` is optional.
9	**Webhooks:** do we use webhooks or polling for shipment updates?	Start with polling every 10–15 minutes (no public endpoint needed), add webhooks later.
10	**Access:** IP allowlist and token scopes.	Register our production server IP. Scopes: `products:read/write`, `orders:read/write`. Add `inventories:read`, `inbounds:write`, `return_orders:read` only when those phases start.
11	**Returns and invoices:** who receives returned goods, and does GW issue invoices?	Returns handled manually first; invoices skipped unless GW issues them.

6. What we have not verified

- No token or sandbox was available, so no call to the live API was made. Everything in section 3 comes from the published documentation.
- The code description in section 2 comes from reading the repository, not from running the Workbench against production data.
- The effort and phasing are estimates; they depend mainly on questions 1, 2 and 4.